Más allá del GPT del libro: codificaciones posicionales para generalizar en longitud y *Mixture of Experts*

María Mestre Sánchez **Elena Fernández Cebrián** **David [apellidos] [nombre] (aos35)**
Procesamiento del Lenguaje Natural I, Grado en Ingeniería en Inteligencia Artificial
Universitat d'Alacant

Resumen

Ampliamos el GPT de los capítulos 2–6 de Raschka (2024) en dos direcciones que tocan módulos distintos del bloque transformer. En la atención, una interfaz de codificaciones posicionales intercambiables (APE aprendida y sinusoidal, NoPE, ALiBi, sesgo relativo de T5 y un híbrido por capas) con la que estudiamos la generalización a longitudes no vistas. En el feed-forward, una capa *Mixture of Experts* (MoE) con router top-k, despacho de tokens por ordenación y pérdida de balanceo de carga. Ambas se validan numéricamente contra PyTorch y Hugging Face (Llama, T5, Mixtral y Switch Transformer) y se evalúan entrenando desde cero en Tiny Shakespeare. ALiBi mantiene la perplejidad hasta 8× la longitud de entrenamiento; el MoE reduce la entropía cruzada en 0.05 nats con el mismo cómputo por token; y, combinados, el MoE mejora con todas las codificaciones pero solo conserva su ventaja al extrapolar con ALiBi: con NoPE, su router concentra en un único experto el 86 % de los tokens de las posiciones no vistas.

1. Introducción

El modelo del libro usa una codificación posicional absoluta aprendida y un feed-forward (FFN) denso. Los LLM actuales sustituyen ambas piezas (Raschka, 2025): codificaciones relativas o ausentes para extrapolar a contextos largos, y capas MoE para multiplicar los parámetros sin multiplicar el cómputo (Shazeer et al., 2017; Lepikhin et al., 2021; Fedus et al., 2022; Jiang et al., 2024). Nuestras contribuciones son: (1) una interfaz de tres ganchos que permite cambiar la codificación posicional sin tocar el resto del modelo (§3); (2) una capa MoE completa con validación contra implementaciones de referencia (§4); (3) la composición de ambas sin modificar el código de ninguna (§5); y (4) una batería reproducible: 111 tests automáticos y 176 autocomprobaciones, y experimentos que se relanzan con un único comando por bloque. RoPE (Su et al., 2024), descartada como ampliación por indicación del profesor, aparece solo como referencia.

2. Modelo base y protocolo

Partimos del GPT *pre-norm* del libro con 4 capas, D=128, 4 cabezas y contexto 128 (≈0.82M parámetros), entrenado desde cero a nivel de carácter en Tiny Shakespeare (Karpathy, 2015) (1.1M caracteres, 65 símbolos, 90/10 entrenamiento/validación) con AdamW (η=10^{-3}, decaimiento 0.1, lotes de 32). Usamos dos protocolos, siempre con 3 semillas y en CPU: **P1** (codificaciones posicionales): 5 épocas sobre ventanas de 128 sin solapar (≈1225 pasos), *dropout* 0.1, y perplejidad de los últimos 32 tokens de ventanas de 32 a 1024 tokens (8× la longitud de entrenamiento); **P2** (MoE): 2000 pasos sobre trozos aleatorios (8.2M tokens), sin *dropout*, y entropía cruzada (CE, en nats) sobre 20 lotes de validación fijos. Las tablas dan media ± desviación típica muestral. Ninguna ampliación modifica `model_base.py`: se activan por configuración y el modelo base sigue disponible como referencia.

3. Codificaciones posicionales

Formulación. Cada codificación implementa tres ganchos con identidad por defecto: `add_to_embeddings` (APE), `apply_qk` (rota q, k; RoPE) y `attn_bias` (sesgo aditivo; ALiBi, T5). La puntuación de la cabeza h entre la consulta i y la clave $j \le i$ es

$$s_{ij}^{h} = \frac{(R_i q_i)^{\top}(R_j k_j)}{\sqrt{d_h}} + b_h(i-j), \qquad (1)$$

seguida de la máscara causal y el softmax. Con APE (aprendida o sinusoidal; Vaswani et al., 2017) R=I, b=0 y se suma p_t al embedding. NoPE no usa nada: el orden solo entra por la máscara causal (Kazemnejad et al., 2023). ALiBi usa $b_h(\Delta)$ =

	CE val	Perplejidad (últimos 32 tokens)			
Codificación	ctx 128	128	256	512	1024
APE aprendida	1.820	6.02	23.73	22.15	–[a]
APE sinusoidal	1.813	5.88	23.17	24.42	23.09
NoPE	2.011	7.50	11.03	13.32	13.94
ALiBi	1.757	5.57	**5.58**	**5.58**	**5.59**
T5	1.772	5.67	6.34	9.40	13.46
RoPE (ref.)	1.677	5.07	13.90	24.57	42.03
Híbrido, N=4	1.698	5.20	15.97	27.09	43.92

Tabla 1: Protocolo P1: entrenadas con contexto 128 y evaluadas hasta 1024 (media de 3 semillas; desviaciones ≤0.011 en CE y ≤1.8 en perplejidad, hasta 6.8 en RoPE y el híbrido). [a]Tabla de 512 posiciones en estas ejecuciones.

$-m_h\Delta$ con $m_h = 2^{-8h/H}$ (Press et al., 2022); T5, un escalar aprendido por cabeza y cubeta de distancia (16 exactas y 16 logarítmicas hasta 128; Raffel et al., 2020). El híbrido `rope_nopeN` quita la codificación a una de cada N capas (Yang et al., 2025).

Validación. La atención con NoPE, ALiBi y RoPE coincide con `scaled_dot_product_attention` de PyTorch (Paszke et al., 2019) (con el sesgo como máscara); RoPE coincide con una referencia en aritmética compleja y con la de Llama en `transformers` (Wolf et al., 2020) (error 0), y las cubetas de T5 con las de Hugging Face. Con los pesos del modelo base, el modelo extendido reproduce sus logits (Tabla 3).

Resultados. La Tabla 1 resume el modelado de lenguaje. Las APE se disparan en cuanto se supera el contexto de entrenamiento (perplejidad ×4). ALiBi es la única que extrapola sin pérdida hasta 8× (5.57→5.59), a cambio de ser algo peor que RoPE dentro de la distribución; T5 degrada gradualmente y NoPE degrada poco en términos relativos pero parte de la peor perplejidad (su CE es 0.25 nats peor que ALiBi). RoPE es la mejor con contexto 128 y la peor a 1024, y el híbrido no lo corrige a esta escala. En tareas algorítmicas (Tabla 2; entrenamiento con $n \le 20$ y evaluación con $21 \le n \le 40$, exact-match) ninguna APE generaliza, NoPE generaliza perfectamente al ordenar (1.00) y ALiBi es la mejor en copiar y en paridad, como en Kazemnejad et al. (2023). Suma, suma módulo 10 y *associative recall* no se aprenden ni en las longitudes vistas (≤0.33) con este presupuesto, así que no permiten comparar.

Tarea	Apr.	Sin.	NoPE	ALiBi	T5	RoPE
Copiar	0.00	0.00	0.21	**0.50**	0.14	0.04
Invertir	0.00	0.00	**0.29**	0.13	0.14	0.05
Ordenar	0.00	0.22	**1.00**	0.49	0.79	0.10
Paridad	0.51	0.50	0.52	**0.61**	0.47	0.49

Tabla 2: Exact-match en longitudes no vistas (media de 3 semillas; en las vistas es 1.00 salvo paridad, 0.67–0.99).

4. Mixture of Experts

Formulación. Cada FFN se sustituye por E expertos con la misma forma y un router lineal. Para un token $x \in \mathbb{R}^D$:

$$p = \mathrm{softmax}(W_r x) \in \mathbb{R}^E, \qquad \mathcal{T} = \mathrm{top}_k(p),$$
$$y = \textstyle\sum_{e\in\mathcal{T}} w_e \,\mathrm{FFN}_e(x). \tag{2}$$

Con k>1, $w_e = p_e / \sum_{\mathcal{T}} p$ (Mixtral); con k=1, $w_e = p_e$ (Switch), porque renormalizar daría $w \equiv 1$ y el router no recibiría gradiente de la CE (medido: $2 \cdot 10^{-6}$ frente a 19.6). Para evitar el colapso en pocos expertos se añade la pérdida de balanceo de Fedus et al. (2022),

$$\mathcal{L}_{\mathrm{aux}} = E \textstyle\sum_{i=1}^{E} f_i P_i, \qquad \mathcal{L} = \mathrm{CE} + \alpha\, \overline{\mathcal{L}_{\mathrm{aux}}}, \tag{3}$$

donde f_i es la fracción de las Nk asignaciones que recibe el experto i (no derivable) y P_i su probabilidad media (por donde entra el gradiente); vale 1 con reparto uniforme y E con colapso total, y se promedia entre capas. **Despacho.** Las $N{=}BT$ filas de índices (N, k) se aplanan en Nk asignaciones, se ordenan por experto (`argsort` estable) y `bincount` da el tamaño de cada bloque; se copian las entradas en ese orden ($Nk{\times}D$), cada experto procesa su bloque contiguo con una sola multiplicación y `index_add_` devuelve cada resultado ponderado a su token, sumando los k de cada uno. Así el coste crece con k y no con E.

Validación. La Tabla 3 resume la batería. Con los pesos copiados, nuestra capa reproduce `MixtralSparseMoeBlock` (con expertos SwiGLU) y `SwitchTransformersSparseMLP`, y nuestra $\mathcal{L}_{\mathrm{aux}}$ es la de Hugging Face dividida por k: normalizamos f para que 1 signifique equilibrio con cualquier k. Con E=1 la capa es el FFN del libro, y un GPT cuyos expertos son copias del FFN denso (Komatsuzaki et al., 2023) reproduce los logits del modelo denso: la integración no altera el modelo base.

Resultados. Con el mismo cómputo por token (k=1), el MoE baja la CE en 0.050 nats (las 3 se-

Comprobación (referencia)	Error máx.
Atención NoPE/ALiBi/RoPE (SDPA, PyTorch)	$\leq 10^{-5}$
RoPE (Llama, HF); cubetas de T5 (T5, HF)	0
Modelo extendido: APE apr., sinusoidal, NoPE (GPT del libro)	$\leq 10^{-6}$
MoE top-k renormalizado (Mixtral, HF)	$8 \cdot 10^{-7}$
MoE top-1 sin renormalizar (Switch, HF)	0
$\mathcal{L}_{\text{aux}}$ y z-loss (HF; $\mathcal{L}_{\text{aux}}^{\text{HF}} = k\,\mathcal{L}_{\text{aux}}$)	$\leq 10^{-6}$
Despachos `sorted`/`loop` (denso; gradientes)	$\leq 10^{-5}$
E=1 (FFN del libro); k=E (mezcla softmax)	$\leq 10^{-5}$
GPT con MoE *upcycled* (denso, 6 codif.)	$\leq 10^{-5}$
Inyección de $\mathcal{L}_{\text{aux}}$ (CE+$\alpha\mathcal{L}_{\text{aux}}$ explícita)	$\leq 10^{-6}$

Tabla 3: Validación numérica con `torch.allclose` (HF = Hugging Face `transformers`).

Modelo	Parám. (act.)	CE val	tok/s	Carga
Denso (libro)	0.82M	1.645±.008	18.6k	–
MoE k=1	2.41M (0.83M)	1.595±.012	17.6k	0.34
MoE k=2	2.41M (1.35M)	1.581±.006	11.9k	0.31
Denso ancho	2.40M	1.576±.004	6.4k	–
k=1, α=0	2.41M (0.83M)	1.605±.006	17.5k	0.65
k=1, α=0.1	2.41M (0.83M)	1.594±.004	17.9k	0.26
k=1, z-loss	2.41M (0.83M)	1.596±.010	16.0k†	0.35

Tabla 4: Protocolo P2 con E=4 y α=0.01 salvo indicación; el denso ancho tiene un FFN 4× más ancho. Carga: fracción de tokens del experto más usado en la peor capa (0.25 = equilibrio); z-loss con coeficiente 10^{-3}. †Medido en otra sesión: no comparable.

millas del MoE mejoran a las 3 del denso) perdiendo solo un 5.6 % de velocidad (Tabla 4). Frente a un denso con los mismos parámetros totales y 4× más cómputo en el FFN, recupera el 72 % de su mejora entrenando 2.75× más rápido; con k=2, el 93 % a 1.86× (la diferencia restante, 0.005, está dentro del ruido). Sin $\mathcal{L}_{\text{aux}}$ el experto más usado llega al 65 % de los tokens (Figura 1c) y la CE empeora ligeramente (tendencia, no significativa con 3 semillas); con α=0.1 el reparto es casi perfecto (26 %) sin coste en CE. La router z-loss (Zoph et al., 2022) reduce un orden de magnitud la escala de los logits del router ($\mathbb{E}[\text{logsumexp}^2]$: 9.2→0.87) y algo su confianza (0.74→0.70), sin cambiar la CE ni el reparto: a esta escala la saturación del router no limita el entrenamiento. En una capa aislada (4096 tokens, *forward+backward*), el despacho por ordenación cuesta lo mismo que el FFN denso (24.1 frente a 23.6 ms) y no depende de E (25.9 ms con E=8), mientras que la referencia densa cuesta E veces más (99 y 203 ms). Los expertos se especializan más con la profundidad: la información mutua normalizada entre el tipo de carácter y el experto sube de 0.08–0.22 en la capa

	CE val (ctx 128)			Perpl. 1024	
Codificación	denso	MoE	Δ	denso	MoE
APE aprendida	1.831	1.772	−0.059±.010	21.66	24.45
NoPE	2.009	1.971	−0.038±.024	13.74	15.89
ALiBi	1.759	**1.694**	−0.065±.007	5.60	**5.17**
T5	1.772	1.723	−0.049±.006	13.87	15.01

Tabla 5: Protocolo P1 con FFN denso y MoE (E=4, k=1, α=0.01, inyectada): 3 semillas emparejadas por codificación. ΔCE: MoE menos denso con la misma semilla.

1 a 0.37–0.41 en la 4, donde un experto recibe el 100 % de los saltos de línea, el 82 % de las mayúsculas y el 86 % de la puntuación.

5. Combinación de ambas ampliaciones

La codificación posicional actúa en la entrada y en la atención y el MoE en el FFN, así que se componen sustituyendo `block.ff` en el modelo con codificaciones intercambiables. El bucle de entrenamiento del protocolo P1 solo optimiza la CE; en lugar de modificarlo, cada capa MoE inyecta $\alpha\,\partial\mathcal{L}_{\text{aux}}/\partial\theta$ en el *backward* mediante una función de autograd que es la identidad en el *forward* (la técnica de Megatron-LM; Shoeybi et al., 2019). Los gradientes coinciden con los de optimizar CE+$\alpha\mathcal{L}_{\text{aux}}$ de forma explícita (Tabla 3).

Resultados. Repetimos el protocolo P1 con FFN denso y MoE para cuatro codificaciones, emparejando semillas en una misma máquina (Tabla 5, Figura 1a). El MoE mejora la CE con las cuatro (de -0.038 a -0.065 nats; las 12 comparaciones emparejadas le favorecen), pero la extrapolación la sigue decidiendo la codificación: con ALiBi la mejora se conserva intacta hasta 1024 (5.60→5.17), con la APE aprendida y NoPE el MoE extrapola peor que el denso en las tres semillas, y con T5 el efecto no es consistente. La Figura 1b apunta a un mecanismo en NoPE: el router de la capa 2 (la 3 en una semilla), equilibrado en las posiciones vistas, asigna el 86 % de los tokens de las posiciones 896–1023 a un único experto (con ALiBi, el 29 %). El logit de ese experto crece con la posición (correlación 0.52, frente a 0.02 con ALiBi): la señal de posición absoluta que NoPE reconstruye a partir de la máscara causal (Kazemnejad et al., 2023) sale de su rango y, como el enrutado top-1 es discreto, subir su probabilidad media de 0.24 a 0.40 basta para reasignar casi todos los tokens. Nuestras ejecuciones densas

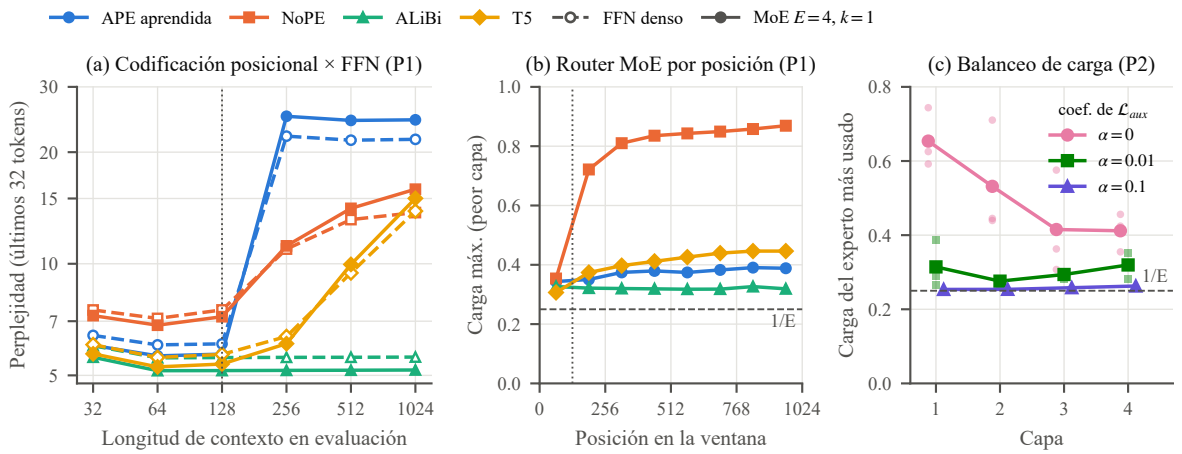


Figura 1: (a) Perplejidad frente a la longitud de contexto con FFN denso (discontinua) y MoE (continua); la línea punteada marca el contexto de entrenamiento. (b) Fracción de tokens del experto más usado (peor capa) por tramo de 128 posiciones en los modelos MoE. (c) Ablación del coeficiente de $\mathcal{L}_{\text{aux}}$ por capa (puntos: semillas). Medias de 3 semillas.

reproducen las medias del equipo con diferencias $\leq$0.011 nats, aunque no semilla a semilla (su modelo evaluado con nuestro código sí da su cifra exacta); por eso solo comparamos ejecuciones emparejadas.

6. Conclusiones y limitaciones

Las dos ampliaciones atacan limitaciones distintas del modelo del libro y se componen sin tocar el código de ninguna. La codificación posicional decide si el modelo extrapola: ALiBi mantiene la perplejidad hasta 8× la longitud de entrenamiento y NoPE ordena perfectamente secuencias más largas que las vistas. El MoE, validado contra Mixtral y Switch, añade capacidad a cómputo constante: -0.05 nats con k=1 y entre el 72 y el 93 % de la mejora de un modelo denso con los mismos parámetros, de 1.9 a 2.75× más rápido. Al combinarlas, el MoE mejora con todas las codificaciones pero solo conserva su ventaja al extrapolar cuando la codificación mantiene estables las representaciones (ALiBi); con NoPE, el router actúa como un detector de representaciones fuera de distribución y concentra la carga en un experto. **Limitaciones:** modelos diminutos entrenados en CPU con un único corpus por caracteres; presupuesto fijo, no convergencia; 3 semillas (diferencias menores de ∼0.01 nats no son concluyentes); tres tareas sintéticas no se aprenden ni en las longitudes vistas; y los tiempos son de CPU (en GPU el desequilibrio de carga es más costoso). Quedan como trabajo futuro la capacidad máxima por experto, los expertos compartidos (Dai et al., 2024) y un enrutado robusto a la posición.

Reproducibilidad. Todo está en la rama `feat/moe` del repositorio del grupo. Los tests (`pytest`) y las autocomprobaciones (`selftest`) se lanzan con un comando, igual que cada bloque de experimentos, y todas las cifras salen de los ficheros de `results/` y `runs/`.

Referencias

Damai Dai et al. 2024. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.

Albert Q. Jiang et al. 2024. Mixtral of experts. arXiv:2401.04088.

Andrej Karpathy. 2015. char-rnn. https://github.com/karpathy/char-rnn.

Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. 2023. The impact of positional encoding on length generalization in transformers. In *Advances in Neural Information Processing Systems*, volume 36.

Aran Komatsuzaki, Joan Puigcerver, James Lee-Thorp, Carlos Riquelme Ruiz, Basil Mustafa, Joshua Ainslie, Yi Tay, Mostafa Dehghani, and Neil Houlsby.

2023. Sparse upcycling: Training mixture-of-experts from dense checkpoints. In *International Conference on Learning Representations*.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*.

Adam Paszke et al. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32.

Ofir Press, Noah A. Smith, and Mike Lewis. 2022. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67.

Sebastian Raschka. 2024. *Build a Large Language Model (From Scratch)*. Manning.

Sebastian Raschka. 2025. The big LLM architecture comparison. Ahead of AI (blog), https://magazine.sebastianraschka.com.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training multi-billion parameter language models using model parallelism. arXiv:1909.08053.

Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

Thomas Wolf et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

Bowen Yang et al. 2025. Rope to nope and back again: A new hybrid attention strategy. arXiv:2501.18795.

Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. 2022. ST-MoE: Designing stable and transferable sparse expert models. arXiv:2202.08906.